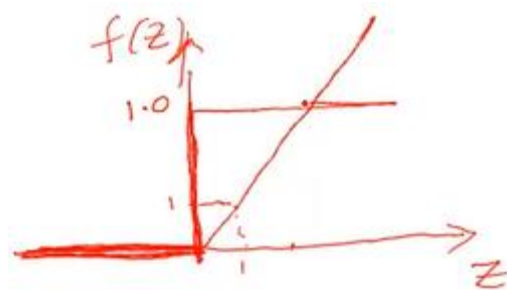
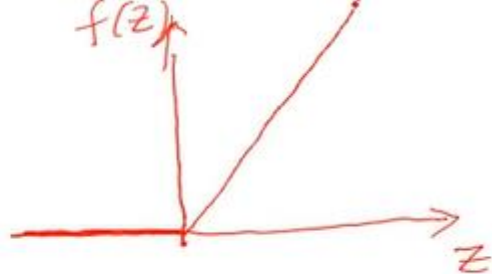


1.0 -50

↓

3) ReLU = $f(z) = 0$ for $z < 0$
 $= z$ for $z \geq 0$

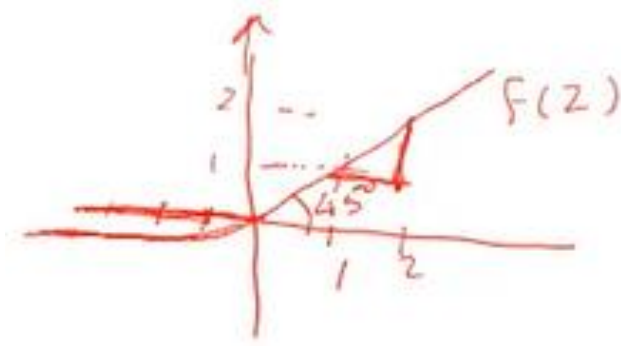
$= \max(z, 0)$



$\frac{\partial \text{Relu}(z)}{\partial z} =$ ~~0 or 1~~
 0 for $z < 0$
 1 for $z \geq 0$

$$f(Z) = \alpha Z \quad \text{for } Z < 0$$

$$= Z \quad \text{for } Z \geq 0$$



$$\tan 45^\circ = 1$$

$$\frac{\partial f(Z)}{\partial Z} = 1$$

$$W_{11}^{new} = W_{11}^{old} - \eta \frac{\partial L}{\partial W_{11}}$$

$$\frac{\partial L}{\partial W'_{11}} = \frac{\partial L}{\partial O_{21}} \times \frac{\partial O_{21}}{\partial O_{11}} \times \frac{\partial O_{11}}{\partial W'_{11}}$$

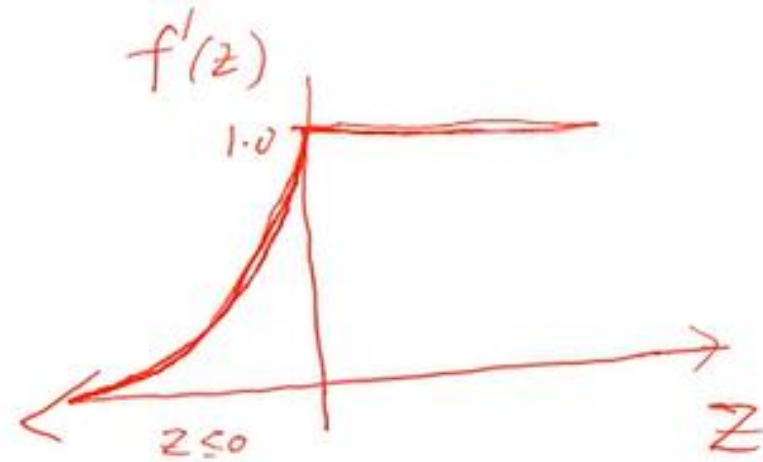
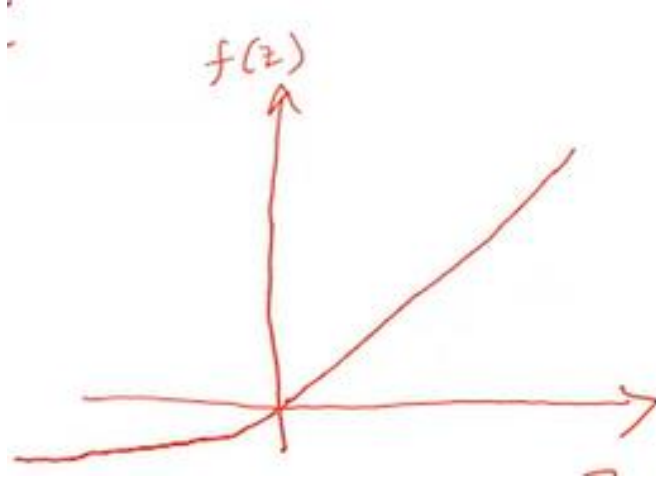
$\alpha_{0.01} \quad \times 0.01$

$$\frac{\partial f(Z)}{\partial Z} = \alpha \quad Z < 0$$

Leaky ReLU

ELU (Exponential Linear Unit)

$$f(z) = \begin{cases} \alpha(e^z - 1) & \text{if } z \leq 0 \\ z & \text{if } z > 0 \end{cases}$$



PReLU (Parametric ReLU)

✓ $f(z) = \begin{cases} \alpha z & \text{if } z < 0 \\ z & \text{if } z \geq 0 \end{cases}$

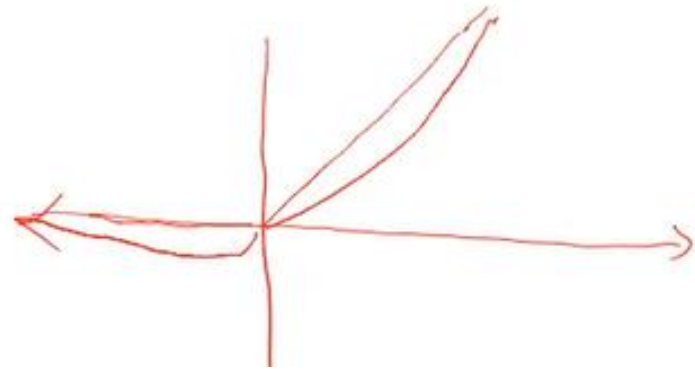
• $\alpha = \underline{0.01} \rightarrow \text{Static} \rightarrow \text{Leaky ReLU}$

$\alpha = 0 \rightarrow \text{ReLU}$

α is any value handle any kind of problem
 $\underline{\alpha} \rightarrow \text{dynamically changing}$

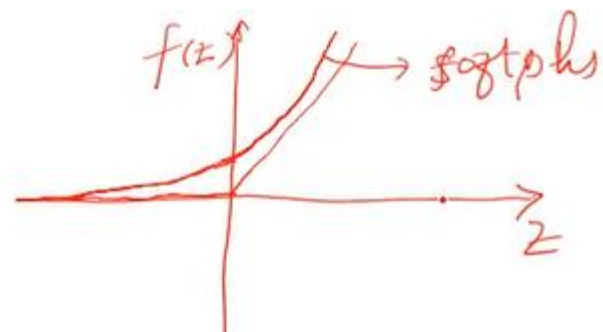
Swish (self-gated)

$$f(z) = z * \text{sigmoid}(z)$$



Softplus:

$$f(z) = \ln(1 + e^z)$$



Softmax Activation

$$f(z) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

where $j = 1, 2, \dots, K$

Dropout Layers in Multi layer Neural Network

Underfitting in NN:

One of the problems that occur during **neural network** training.

Underfitting is the case where the model has “not learned enough” from the training data, resulting in unreliable predictions.

A model with too little capacity cannot learn the problem.

Too little learning will make the model to perform poorly on the training dataset and on new data.

An underfit model has high bias and low variance.

Dropout Layers in Multi layer Neural Network (Contd..)

Overfitting in NN:

One of the problems that occur during **neural network** training is called **overfitting**.

The error on the training set is driven to a very small value, but when new data is presented to the **network** the error is large.

The **network** has memorized the training examples, but it has not learned to generalize to new situations.

A model with too much capacity can learn it too well and overfit the training dataset.

With too much learning, the model will perform well on the training dataset and poorly on new data.

An overfit model has low bias and high variance. The model learns the training data too well and performance varies widely with new unseen. (Draw the graph of accuracy vs epoch)

Good Fit Model. A model that suitably learns the training dataset and generalizes well to the data set.

Dropout Layers in Multi layer Neural Network (Contd..)

How to handle underfitting problem?

Underfitting can easily be addressed by increasing the capacity of the network.

How do you handle overfitting problem?

i) Reduce the network's capacity by removing layers or reducing the number of neuron elements in the hidden layers.

ii) Use **Dropout layers**, which will randomly remove certain features by setting them to zero.

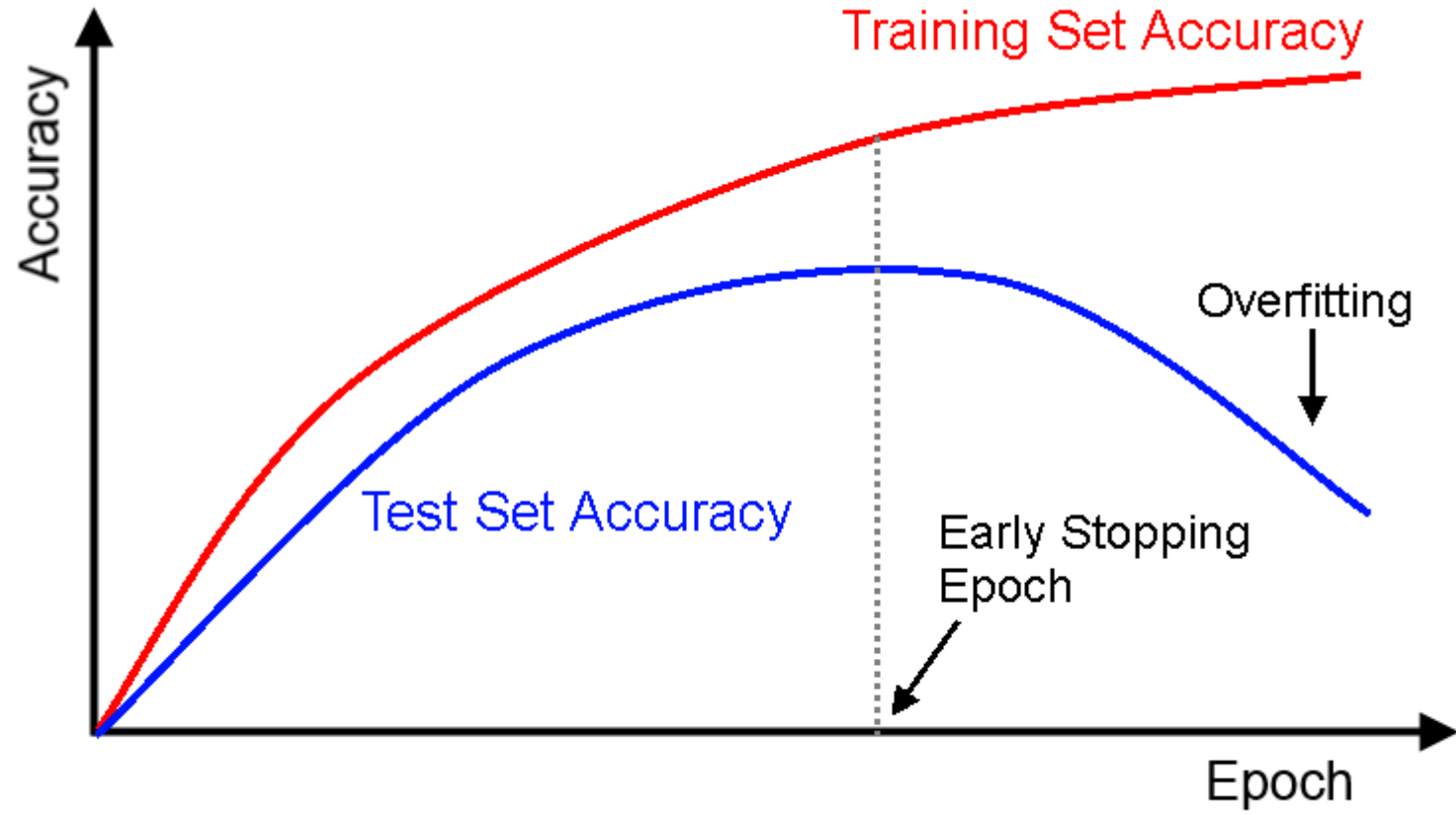
Dropout Layers are an easy and effective **way to prevent overfitting** in NN.

A dropout layer randomly drops some of the connections between layers. This helps to **prevent overfitting**.

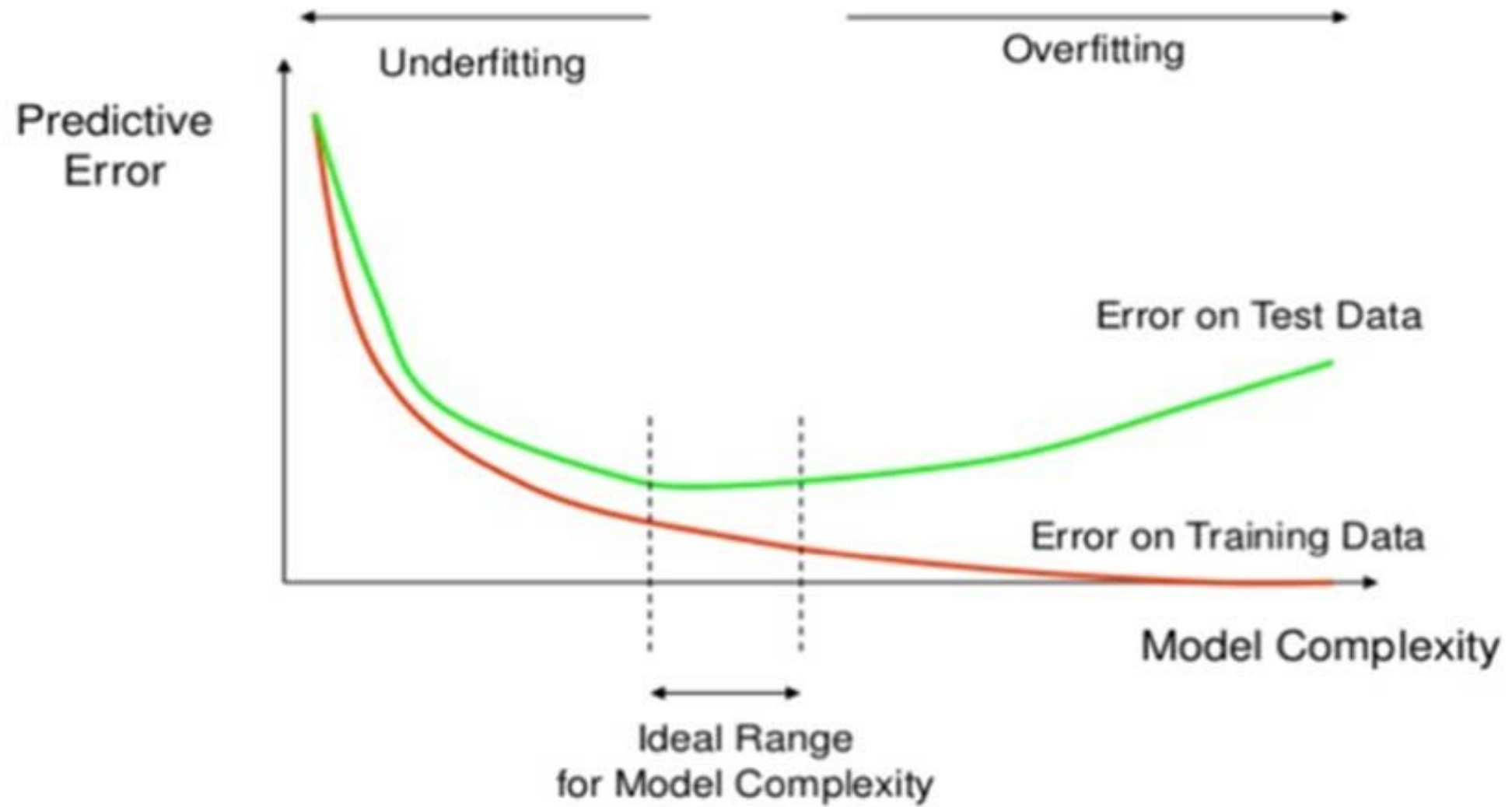
We reduce the weights of the model to small during learning.

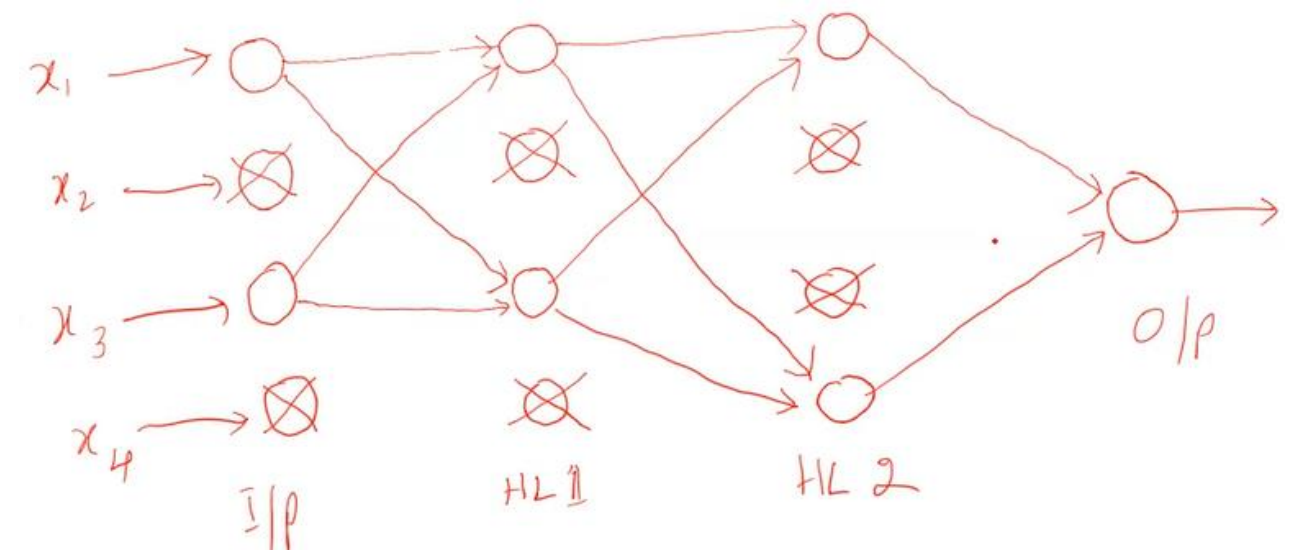
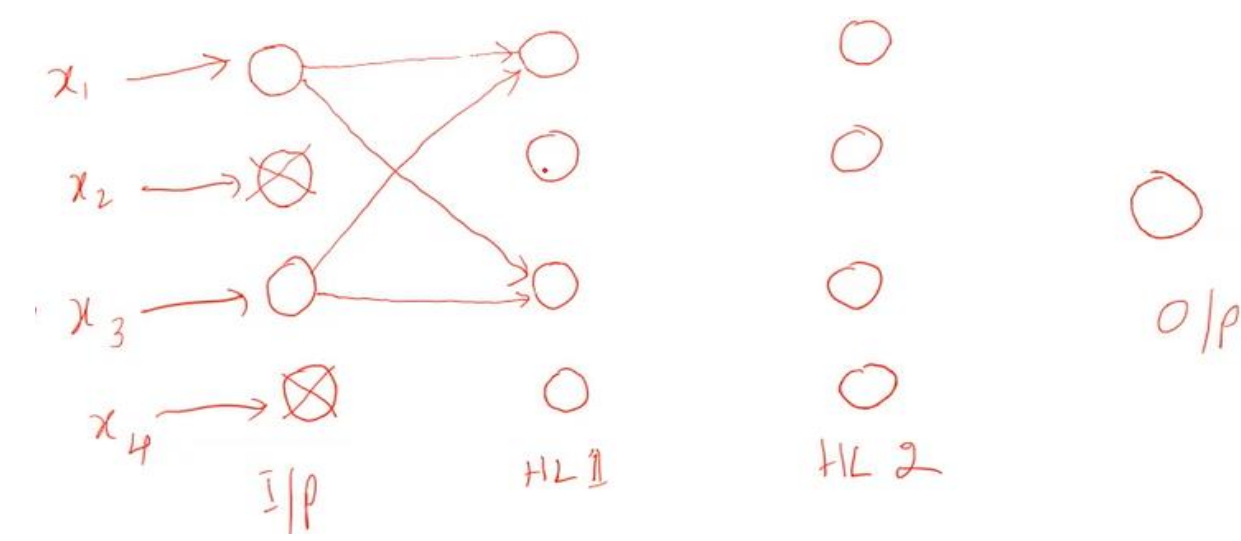
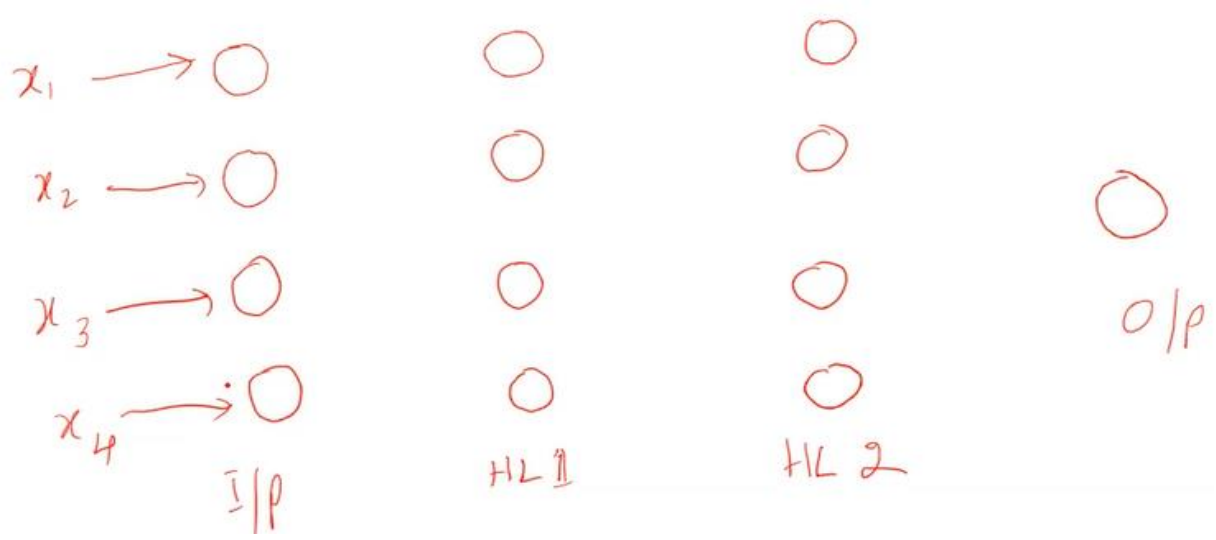
These techniques not only reduce overfitting, but they can also lead to faster optimization of the model and better overall performance.

Dropout layers in Multi-layer NN



Dropout layers in Multi-layer NN





Dropout ratio:

Dropout is a Simple Way to Prevent **Neural Networks** from Overfitting.

Because the outputs of a layer under **dropout** are randomly subsampled, it has the effect of reducing the capacity or thinning the **network** during training.

Weight initialization techniques in NN:

..

Key points to note while init the weights:

- 1) Weights should be small.
- 2) Weights should not be same.
- 3) Weights should have good variance.

Weight initialization techniques in NN

- 1. Uniform distribution technique
- 2. Xavier / Gorat distribution technique
- 3. He initialization technique

1. Uniform distribution technique:

Weights are taken and sampled from uniform distri

W_i will be in the range

$$-\frac{\frac{a}{\sqrt{f_{\text{an_in}}}}}{\sqrt{f_{\text{an_in}}}} \quad \frac{b}{\sqrt{f_{\text{an_in}}}}$$

Weight initialization techniques in NN

-2.Xavier / Gorat distribution technique

i) " " Uniform a, b

w_i will be in range $-\frac{\sqrt{6}}{\sqrt{(\text{fan_in} + \text{fan_out})}}, +\frac{\sqrt{6}}{\sqrt{(\text{fan_in} + \text{fan_out})}}$

ii) Xavier / Gorat normal

w_i in range of $N(\text{mean } 0, \text{std dev } \sigma)$

$$\sigma = \sqrt{\frac{2}{(\text{fan_in} + \text{fan_out})}}$$

Weight initialization techniques in NN

-3. He initialization technique

i) He uniform

w_i in range of

$$a, b \\ -\sqrt{\frac{6}{\text{fan_in}}}, +\sqrt{\frac{6}{\text{fan_in}}}$$

ii) He Normal

$$w_i \sim N(\text{mean } 0, \sigma)$$

$$\sigma = \sqrt{\frac{2}{\text{fan_in}}}$$

Optimizers

- Gradient Descent (GD)
- Stochastic Gradient Descent (SGD)

Global minima / maxima, Local minima / maxima concepts

Gradient Descent (GD) $L = (y - \hat{y})^2$

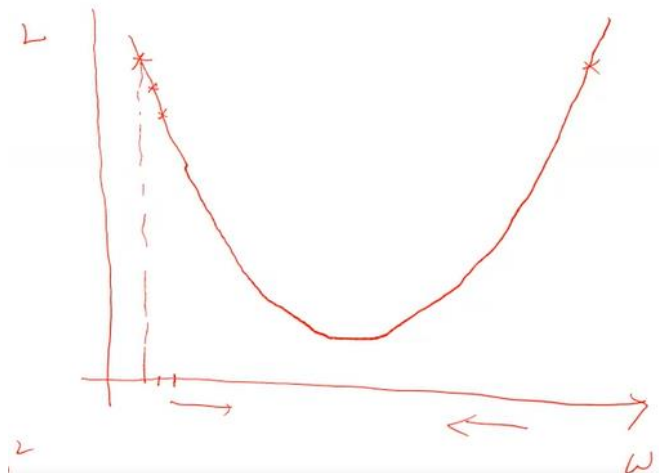
$$\text{Cost}_{fn} = \sum_{i=1}^n (y - \hat{y})^2$$

Stochastic gradient Descent (SGD)

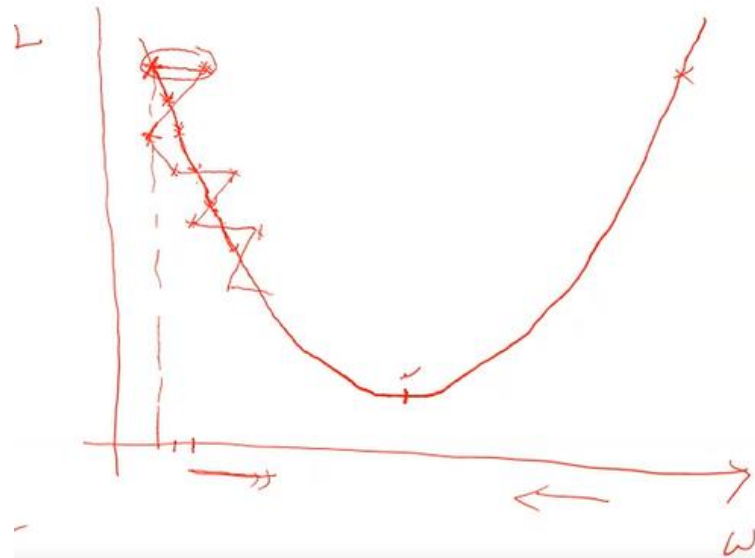
only one data at a time from data set is considered to update a parameter

Variation:

a subset of data from " " " " " " "

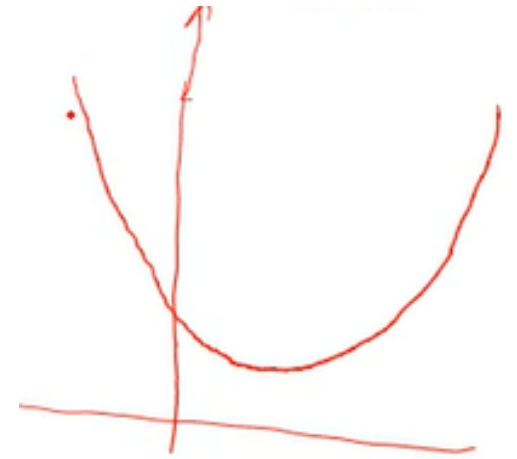
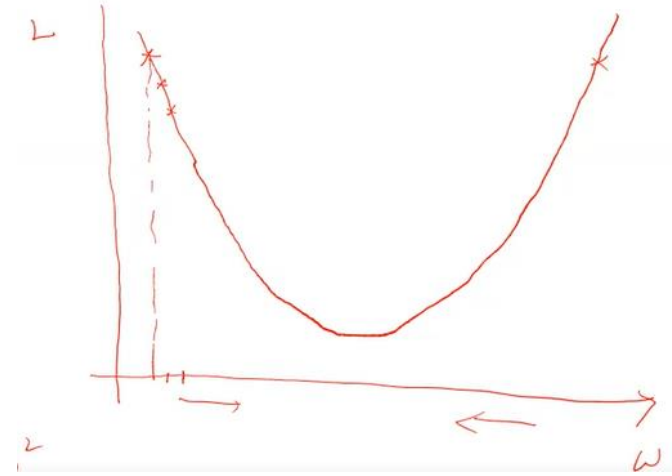
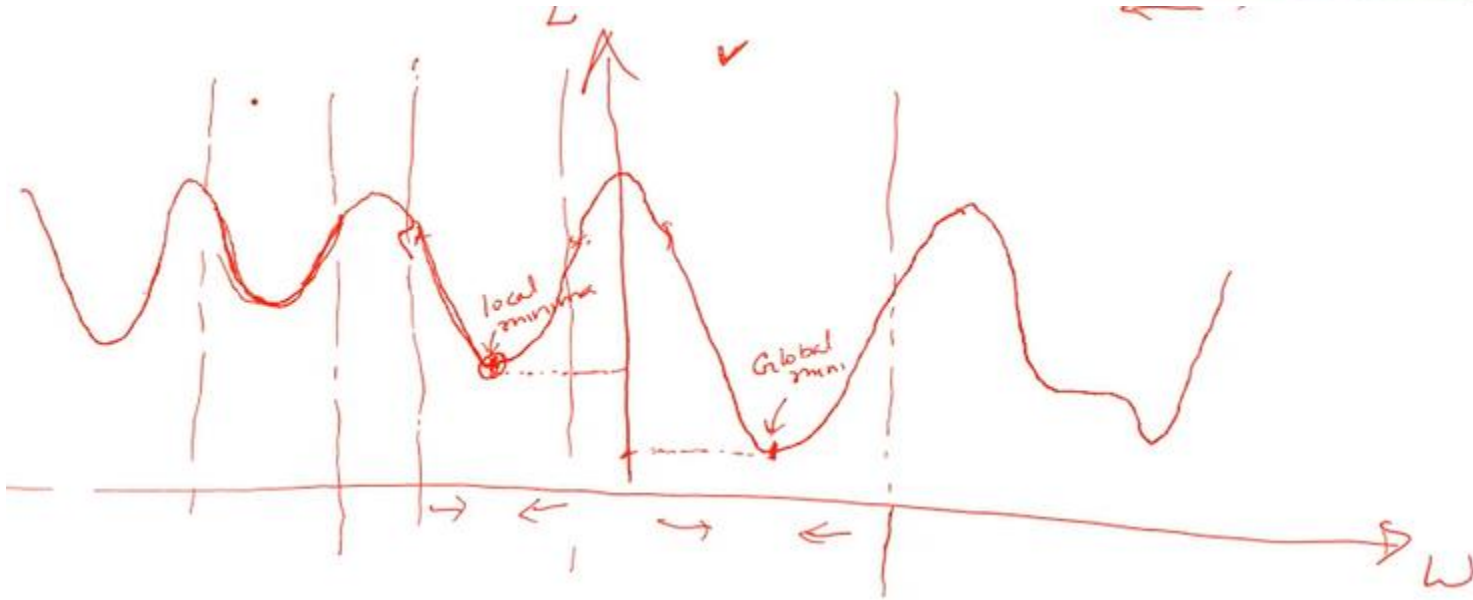
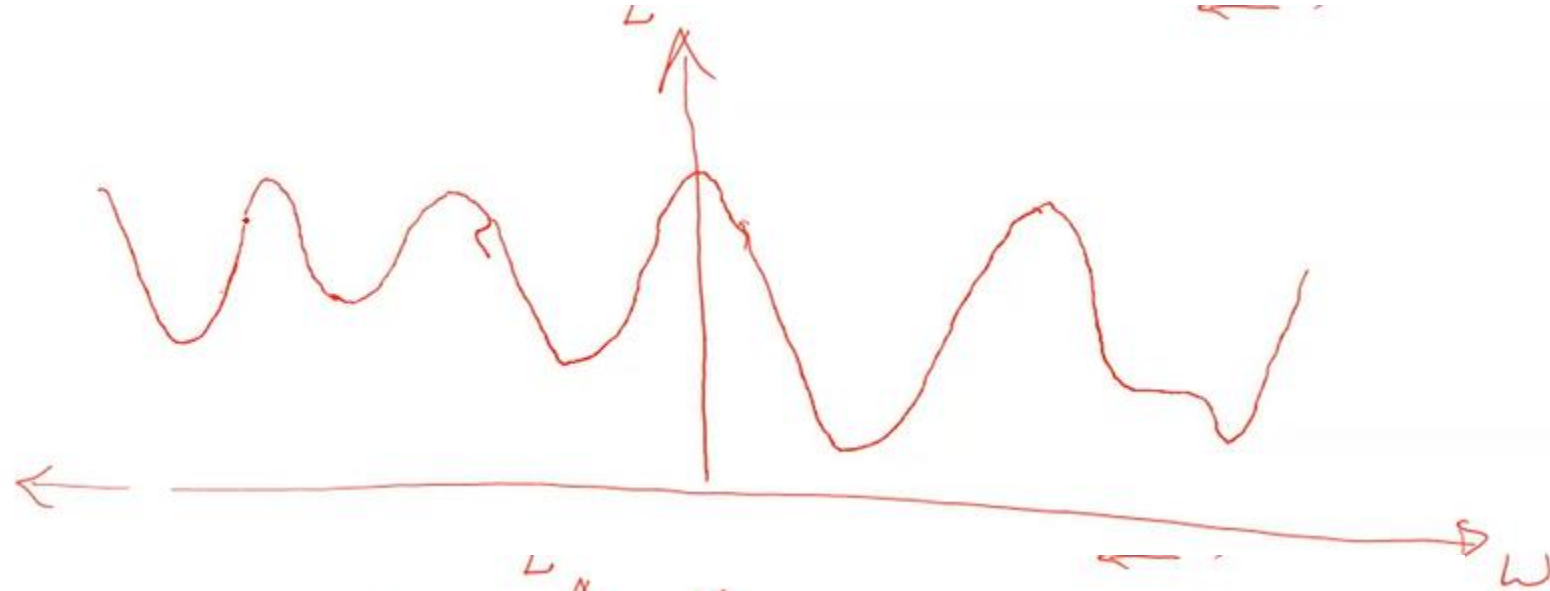


GD



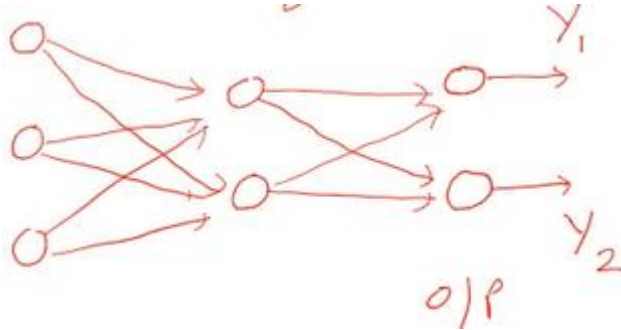
SGD

Global minima / maxima, Local minima / maxima concepts



Highlights on Error / Loss calculation in NN

$$\textcircled{L} = (y - \hat{y})^2 \checkmark$$
$$\text{Cost} = \sum_{i=1}^n (y - \hat{y})^2$$



$$L = \frac{1}{2} (y_1 - \hat{y}_1)^2$$

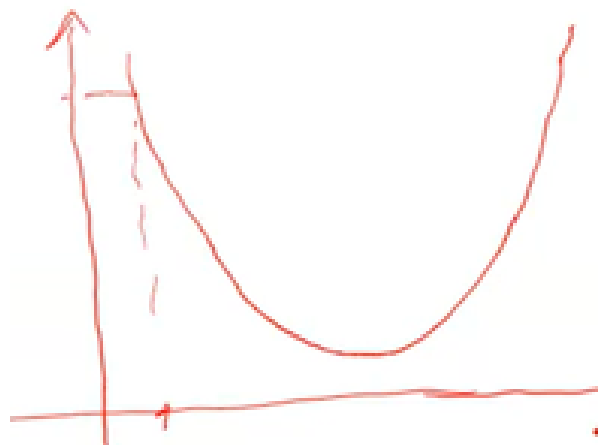
$$L = \frac{1}{2} (y_2 - \hat{y}_2)^2$$

$$\text{Cost} = \frac{1}{2} \sum_{i=1}^n (y - \hat{y})^2$$

If N o/ps are there from a NN then

$$\text{Cost} = \frac{1}{N} \sum_{i=1}^n (y - \hat{y})^2$$

Stochastic Gradient Descent (SGD) with momentum



Exponentially moving weight

time	t_1	t_2	t_3	...	t_n
datapts	b_1	b_2	b_3		b_n

$$V_1 = b_1 \text{ at } t_1$$

$$V_2 = \gamma V_1 + (1-\gamma) b_2 \text{ at } t_2$$

$0 \leq \gamma \leq 1$

$\gamma = 0.05$

$V_2 = 0.05$

0.95

$$V_3 = \gamma V_2 + (1-\gamma) b_3$$

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w_{\text{old}}}$$

$$w_{\text{old}} - \eta V_n$$

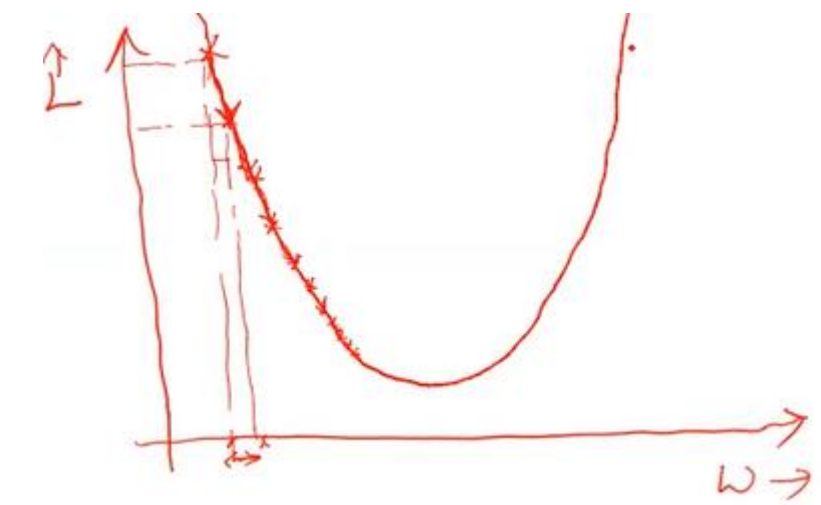
$$= w_{\text{old}} - \eta \left[\gamma V_{n-1} + (1-\gamma) \frac{\partial L}{\partial w} \right]$$

Adagrad optimizer: ..

Adaptive gradient descent optimizer
 $\eta \rightarrow$ dynamically variable
 Every iteration $\eta \rightarrow$ variable

0.01
 -0.009
 -0.008
 ↓
 =

$\rightarrow t_1 \rightarrow \frac{\partial L}{\partial w_1}$
 $\rightarrow t_2 \rightarrow \frac{\partial L}{\partial w_2}$
 $\rightarrow t_3 \rightarrow$
 $\rightarrow t_4 \rightarrow$
 $\rightarrow t_n \rightarrow$



$$w_t = w_{t-1} - \eta \frac{\partial L}{\partial w_{t-1}}$$

$$w_t = w_{t-1} - \eta'_t \frac{\partial L}{\partial w_{t-1}}$$

$$\eta'_t = \frac{\eta}{\sqrt{\alpha_t}}$$

$$\eta'_t = \frac{\eta}{\sqrt{\alpha_t + \epsilon}}$$

If $\alpha_t = 0$
 $\sqrt{0} = 0$

If $\alpha_t = 0$
 $\sqrt{0} = 0$
 $\epsilon \rightarrow$ small +ve number

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial w_i} \right)^2$$

$\alpha_t \uparrow$ $\eta'_t \downarrow$

As $\eta'_t \downarrow$ $w_t \downarrow$ slowly

Benefit of changing the learning rate:
 Disadv: Vanishing gradient problem.

$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial w_i} \right)^2$
 $\uparrow \uparrow \uparrow$ $\eta'_t \downarrow \downarrow \downarrow$
 (0.000000)

Adadelta Optimizer and RMSProp Optimizer:

Drawback in adagrad optimizer is vanishing grad problem can be overcome with adadelta. ..

$$w_{new} = w_{old} - \eta'_t \frac{\partial L}{\partial w_{old}}$$

$$\eta'_t = \frac{\eta}{\sqrt{\cdot} + \epsilon}$$

$$\eta'_t = \frac{\eta}{\sqrt{w_{avg} + \epsilon}}$$

w_{avg} but is not as increase as α_t

$\eta'_t \downarrow$ slowly

$w_{avg} \uparrow$ slow

$\eta'_t \downarrow \downarrow$

$$w_{avg}_t = \gamma \times w_{avg}_{t-1} + (1-\gamma) \left(\frac{\partial L}{\partial w_{t-1}} \right)^2$$

$\gamma = 0.95$
 $0.95 \times \quad + 0.05 \times$

$$\alpha_t = \sum_{i=1}^t \left(\frac{\partial L}{\partial w} \right)^2$$

Adam optimizer

Combine two techniques:

- SGD with momentum optimizer
- RMSProp optimizer

SGD with momentum

$$V_n = \gamma_1 V_{n-1} + (1 - \gamma_1) \frac{\partial L}{\partial \omega}$$

RMSProp

$$w_{avg_t} = \gamma_2 w_{avg_{t-1}} + (1 - \gamma_2) \left(\frac{\partial L}{\partial \omega_{t-1}} \right)^2$$

$$\omega_t = \omega_{t-1} - \frac{\eta \times V_n}{\sqrt{w_{avg_t} + \epsilon}}$$